Adaptive Hash Index

우아한테크코스 8기 BE 제제
Real MySQL Study

쇼핑몰에 취직한 제제

고객 응대 및 상품 진열 정리를 맡았다!

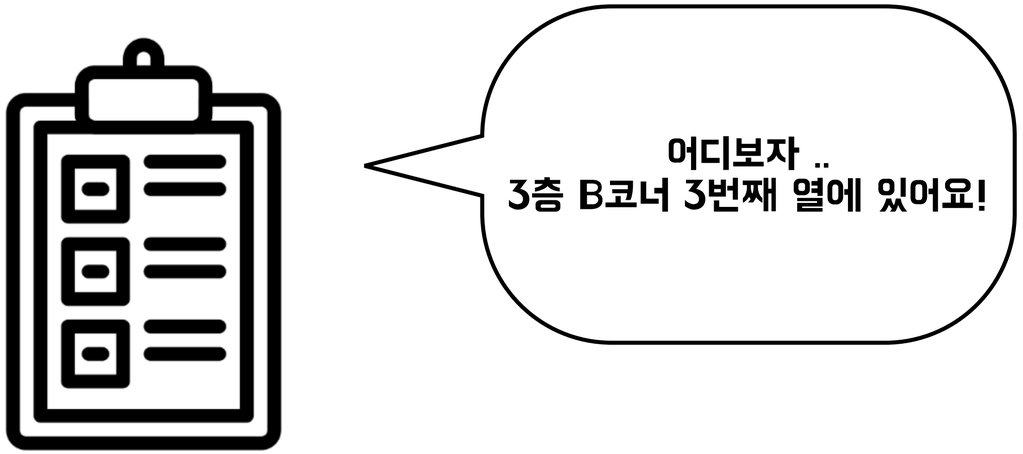
정리를 열심히 하던 어느 날..

쇼핑몰의 '우아한 초콜릿'이 대히트를 치게 됐다!

우아한 초콜릿 어디있어요?

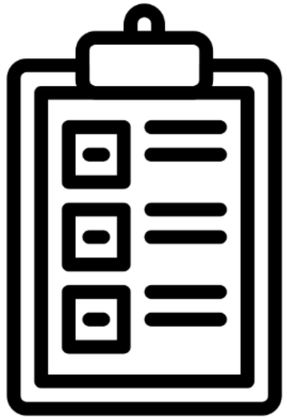
우아한 초콜릿을 찾는 고객들이 제제에게 상품의 위치를 묻는다!

아직 일이 미숙한 제제..
상품 목록을 처음부터 읽어 찾는다



얼마 지나지 않아..
또 다른 고객이 우아한 초콜릿을 찾는다

금붕어 기억력 제제는 다시 상품 목록을 찾아 알려준다



어딨더라 ..
3층 B코너 3번째 열에 있어요!

우아한 초콜릿 어디있어요?

너무 히트한 탓일까..

하루에 50000명의 고객이 우아한 초콜릿을 찾는다!!!

우아한 초콜릿 어디있어요?

도무지 일을 할 수 없게 된 제제
기지를 발휘하는데..

안내문을 작성해 붙여놓았다

안내

우아한 초콜릿
3층 B코너 3번째 열에 있습니다!

안내문의 효과로..

50000명의 고객이 제제에게 상품 위치를 묻지 않게 됐다!

다시 일에 집중할 수 있게된 제제

이번엔 '우아한 쫀득 쿠키'가 히트를 친다 😭

제제는 작성한 안내문에 '우아한 쫀득 쿠키'를 추가한다.

안내

우아한 초콜릿
3층 B코너 3번째 열에 있습니다!

우아한 쫀득 쿠키
1층 C코너 5번째 열에 있습니다!

질문 지옥에서 벗어난 제제는 에이스가 된다.

하루가 다르게 생겨나는 히트아이템

오늘은 '우아한 버터떡'이 히트를 쳤다.
제제는 안내 문구를 수정한다..

우아한 초콜릿
층 B코너 3번째 열에 있습니다!

우아한 쫀득 쿠키
층 C코너 5번째 열에 있습니다!

3층 B코너 3번째 열에 있습니다!

우아한 쫀득 쿠키
1층 C코너 5번째 열에 있습니다!

우아한 초콜릿
3층 B코너 3번째 열에 있

우아한 쫀득 쿠키
1층 C코너 5번째 열에 있

그렇게 제제가 안내 문구에 추가한 아이템은 어느덧 50000개 가 되었다.

아

우아한 초콜릿

안내

우아한 초콜릿

안내

우아한 초콜릿

청천벽력같은 소식!!!

쇼핑몰이 전 층 리뉴얼을 한다고 한다

상품 위치가 모두 바뀌게 되었다.

안내 문구를 수정해야 했다.

너무 많은 탓에..

밤새 바꿔봤지만, 결국 수정하지 못한 제제는
하루 100,000명이 넘는 고객의 질문 세례에
일을 하지 못하고 결국 해고된다.

제제 = CPU(+ InnoDB)
안내문 = 해시 인덱스
손님 = SQL 쿼리

동일한 SQL 쿼리를 처리하느라
제제(CPU)는 제대로 일을 하지 못했다.

그래서 동일한 SQL을 간편히 처리하도록
해시 인덱스를 이용했다!

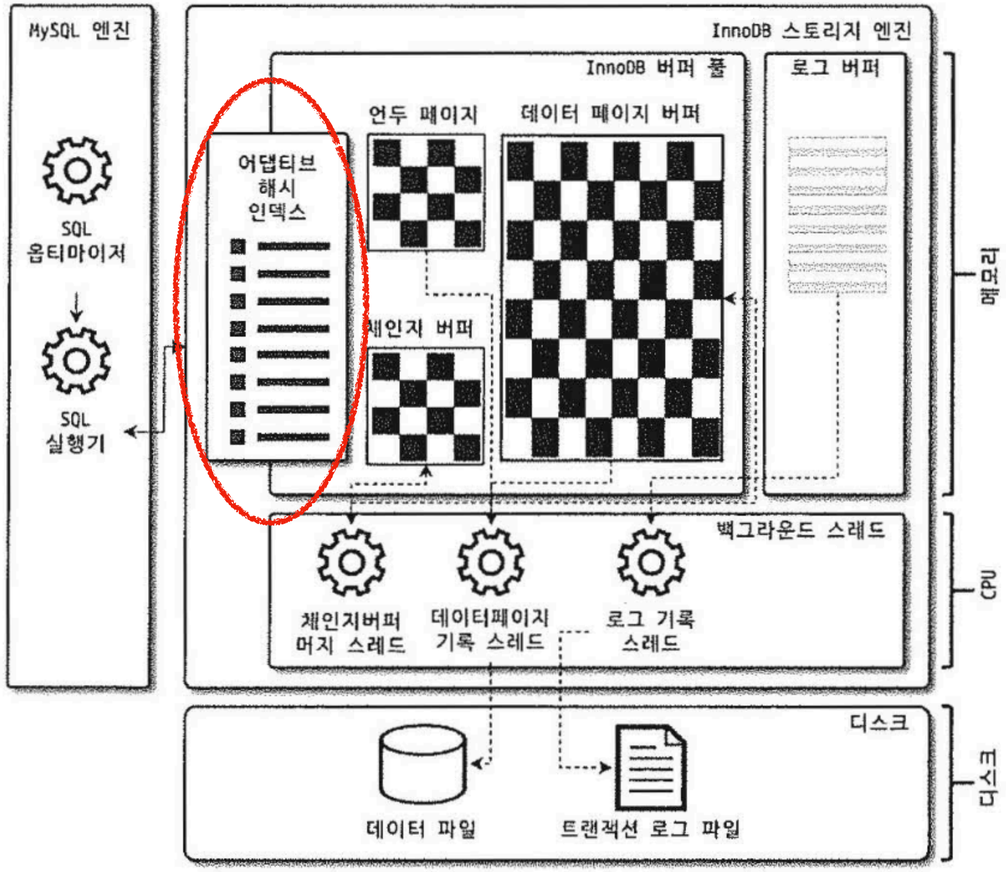
해시 인덱스인거 알겠어.

근데 어댑티브 해시 인덱스는 뭐야?

자주 조회되는 레코드를 InnoDB가 '적응적으로'
해시 인덱스에 추가한다.

어댑티브 해시 인덱스의 구조

여러 테이블의 캐시 값을 한 테이블에서 관리한다.



크루 테이블의 "제제"와
코치 테이블의 "검프"가
같은 해시 테이블에서 관리된다.

키와 값은 어떻게 구성되나?

키 = 테이블 고유번호 + ID
값 = "InnoDB 버퍼 풀"에 위치한 데이터 주소

예시) '크루' 테이블의 ID 5번인 '제제'를 해시 테이블에 추가한다면?
-> (CREW5, 0x000..) 형태로 저장!

중요한 점: '디스크상 위치'를 저장하는 게 아니다!
'InnoDB 버퍼 풀'에 저장된 위치를 표시한다.
즉, InnoDB 버퍼 풀에서 사라지면 해시 인덱스에서도 사라진다!

어댑티브 해시 인덱스의 장점

1. 자주 사용되는 레코드를 O(1)로 찾을 수 있다.
2. 디스크 조회 횟수가 줄어들어 성능이 향상된다

어댑티브 해시 인덱스의 단점

1. 데이터가 변경될 경우(예시처럼) 해시 인덱스도 동시에 변경되어야 한다.
   -> 쓰기 작업이 많은 환경에서는 오히려 독이 될 수 있다.
2. InnoDB 버퍼 풀 메모리 일부를 사용하므로 데이터 캐싱 공간이 줄어든다.
3. 동등 비교 조건(=, IN)만 사용할 수 있다. LIKE나 <, > 등의 연산자는 사용할 수 없다.

끗!